

420 Hours in Claude Code: The Three Lessons

A list of its own mistakes that has to become code, a grader that never built the thing, and a greeting that leads with what is working. Each one explained in plain English, plus one prompt that sets all three up.

Built by [@brycenwood.ai](#) · September 2026

WHY THESE THREE

420 hours in Claude Code since March, running a vehicle wrap shop and a meal prep company with it. Not tutorials. Invoices, customer texts, cook lists. These are the three lessons that took the longest to learn, and none of them are tips. They are how the thing gets smarter about itself, and how it shapes you back.

ONE

It keeps a list of its own mistakes

Every AI forgets what it got wrong. So mine writes each mistake into one file the moment it happens: what went wrong, and what that mistake has to turn into. A hook that blocks it, a test that catches it, or a scheduled check that reports it. Thirty-six entries so far. A mistake that is still just a note after one session was not learned.

How: a file called `lessons.json` and one rule in `CLAUDE.md`: *When you get something wrong, add it to lessons.json the same turn with what happened and what it must become. Build that thing before the session ends and mark it implemented.*

The one that builds never grades

Whoever writes a plan cannot see its own blind spot. So a second Claude, a brand new process with no memory of the work, reads the plan file cold and grades it. Read only, never fix. Mine took a plan from a D+ to an A- in five passes and every pass found something the author swore was fine.

How: a two-line script, `grade.sh`, that starts a fresh `c\laude -p` with read-only tools: *Read the file at this path. Grade it A to F. List every claim you cannot verify, every number with no source, and every step that cannot be run as written. Do not fix anything.* Then the rule: *Before I approve any plan you wrote, run grade.sh on it and show me the findings word for word.* Give it the path, never a summary.

THREE

The greeting shapes who you think you are

A SessionStart hook runs the second you open Claude Code, before you type. Mine used to print everything I had not finished. Every morning. That was feeding a story about myself I am trying to get out from under. Now it prints what is working first, then what needs me. Same information, opposite effect.

How: a hook in `.c\laude/settings.json` under SessionStart that runs a small script: `WORKING`, three things shipped or running from the last week, then `NEEDS ME`, at most three. Plus one rule in `CLAUDE.md`: *Open every session with what is working before what is unfinished. Never read the to-do list back to me as a greeting.*

THE PROMPT

One paste sets up all three

Open the terminal inside the folder Claude Code already runs from and type `c\laude`. Paste this and answer its questions one at a time. It shows you everything it built at the end and runs the new greeting once.

PASTE THIS INTO CLAUDE CODE

Set up three things in this folder. Ask me one question at a time when you need something from me, and explain what each thing DOES before

how it works.

1. A LIST OF YOUR OWN MISTAKES. Create a file called `lessons.json`. Add this rule to `CLAUDE.md`: "When you get something wrong, or I correct you, add an entry to `lessons.json` the same turn with: what happened, the date, and what it must become: a hook that blocks it, a test that catches it, or a scheduled check that reports it. Then build that thing before the session ends and mark the entry implemented. An entry that is still only a note after one session means it was not learned." Then ask me for the last three times you or any AI got something wrong for me, and write them in as the first three entries, each with what it should become.

2. THE BUILDER NEVER GRADES. Create a short script called `grade.sh` that takes a file path and runs a brand new Claude Code process with read-only tools to grade it: `claude -p "Read the file at $1. Grade it A to F. List every claim you cannot verify, every number that has no source, and every step that cannot be run as written. Do not fix anything, only report." --allowedTools "Read,Glob,Grep"`. Add this rule to `CLAUDE.md`: "Whoever writes a plan never grades it. Before I approve any plan you wrote, run `grade.sh` on the file and show me the grade and the findings word for word. Fix, regrade, repeat until it is an A or I say stop." Then explain to me, in plain English, why a fresh process with no memory of the work finds things you cannot.

3. THE GREETING. Set up a `SessionStart` hook that runs a small script every time I open Claude Code here, before I type anything. It prints two short blocks: `WORKING`, three things that are running or shipped, from the last week of git history and anything in a file called `working.md`, and then `NEEDS ME`, at most three things. Working always prints first. Add this rule to `CLAUDE.md`: "Open every session with what is working before what is unfinished. Never read the to-do list back to me as a greeting." Ask me what I want the `WORKING` block to pull from if there is no git history yet.

When all three are done, show me `lessons.json`, `grade.sh`, the hook, and the new lines in `CLAUDE.md`, and run the greeting once so I can see it.

That is the whole system.

Zero ad spend. Zero coding experience. The other playbooks live at brycenwood.com/playbook, one download each.

Built by Brycen Wood · [@brycenwood.ai](https://twitter.com/brycenwood.ai) · brycenwood.com